

Estudio: PythonChatGPT

Índice

1. [¿Qué hace este proyecto?](#)
 2. [Arquitectura general](#)
 3. [Flujo de ejecución](#)
 4. [Desglose de cada archivo](#)
 5. [Dependencias externas](#)
 6. [Conceptos clave](#)
 7. [Posibles mejoras](#)
-

1. ¿Qué hace este proyecto?

Es un **chatbot de terminal** (CLI) que se conecta a un modelo de lenguaje ejecutándose localmente a través de [Ollama](#). El usuario escribe preguntas y el modelo responde en tiempo real (streaming), manteniendo el contexto de toda la conversación.

Tecnologías involucradas:

- Ollama (servidor local de LLMs)
 - API compatible con OpenAI (el SDK oficial de OpenAI)
 - Terminal interactiva con formato visual (Rich + Typer)
-

2. Arquitectura general

```
main.py ← único punto de entrada
├── Lee variables de entorno (OLLAMA_URL, OLLAMA_API_KEY, OLLAMA_MODEL)
├── Crea cliente OpenAI apuntando a Ollama
└── Bucle principal:
    ├── Muestra menú (Rich Table)
    ├── Lee input del usuario
    ├── Envía mensajes a Ollama con streaming
    ├── Muestra respuesta token por token
    └── Acumula historial de la conversación
```

No hay separación en capas — todo el código vive en un solo archivo (`main.py`). Esto es típico de prototipos y proyectos pequeños, pero para crecimiento conviene dividir en:

- `config.py` — configuración
 - `client.py` — comunicación con la API
 - `chat.py` — lógica de conversación
 - `cli.py` — interfaz de terminal
-

3. Flujo de ejecución

3.1 Importaciones (líneas 1-6)

```
from openai import OpenAI      # SDK oficial de OpenAI
from openai import APIConnectionError # Excepción específica
import typer                  # Framework para crear CLIs
from rich import print        # print con colores y formato
from rich.table import Table  # Tablas formateadas en terminal
import os                    # Leer variables de entorno
```

openai: SDK oficial. Aunque está diseñado para la API de OpenAI, también funciona con servidores compatibles como Ollama (que expone una API idéntica). Esto permite cambiar entre Ollama local y OpenAI en la nube sin tocar el código.

typer: Framework para crear aplicaciones de línea de comandos. Convierte una función Python en un comando CLI con `--help` automático.

rich: Biblioteca para terminales bonitas. Permite colores, tablas, markdown, barras de progreso, etc.

3.2 Configuración desde entorno (líneas 8-12)

```
BASE_URL = os.getenv("OLLAMA_URL", "http://localhost:11434/v1")
API_KEY = os.getenv("OLLAMA_API_KEY", "ollama")
MODEL = os.getenv("OLLAMA_MODEL", "llama3.2:1b")

client = OpenAI(base_url=BASE_URL, api_key=API_KEY)
```

`os.getenv(var, default)` busca la variable de entorno `var` . Si no existe, usa `default` .

Por qué `api_key="ollama"` : Ollama no requiere autenticación real, pero el SDK de OpenAI exige una API Key. Se pasa cualquier valor.

`base_url` : La URL de la API. Ollama por defecto expone su API en `http://localhost:11434/v1` , que es compatible con el formato de OpenAI.

3.3 La función main() (líneas 15-52)

3.3.1 Cabecera (líneas 16-20)

```
def main() -> None:
    print("[bold yellow]Chat-bot con IA (Ollama)[/bold yellow]")

    table = Table("Opciones", "Descripción")
    table.add_row("Salir", "Escribir 'salir' para salir del programa")
    print(table)
```

`[bold yellow]...[/bold yellow]` es sintaxis de Rich para aplicar formato.

`Table("Opciones", "Descripción")` crea una tabla de 2 columnas. `add_row()` agrega una fila.

3.3.2 Mensaje de sistema (líneas 22-24)

```
messages = [
    {"role": "system", "content": "Eres un asistente muy util."}
]
```

Esta es la **base del protocolo de chat de OpenAI** (y Ollama). Cada mensaje tiene:

- `role` : puede ser `"system"`, `"user"`, o `"assistant"`
- `content` : el texto del mensaje

El mensaje de sistema define la **personalidad y comportamiento del asistente**. Se envía al inicio y el modelo lo usa como instrucción general.

3.3.3 Bucle principal (líneas 26-52)

```
while True:
    content = input("Escribe tu pregunta: ")
    if content == "salir":
        break
    messages.append({"role": "user", "content": content})
```

`input()` : Función nativa de Python que muestra un prompt y espera texto del usuario.

Se agrega el mensaje del usuario al historial (`messages`).

3.3.4 Llamada a la API con streaming (líneas 32-45)

```
try:
    response = client.chat.completions.create(
        model=MODEL,
        messages=messages,
        stream=True,
    )

    assistant_reply = ""
    for chunk in response:
        if chunk.choices[0].delta.content:
            content_chunk = chunk.choices[0].delta.content
            print(content_chunk, end="")
            assistant_reply += content_chunk
    print()
```

stream=True : Activa el streaming. En lugar de esperar a que el modelo termine de generar toda la respuesta, la API devuelve **tokens individuales** (o fragmentos) a medida que se generan.

client.chat.completions.create() : El método principal del SDK de OpenAI. Parámetros:

- **model** : nombre del modelo en Ollama (ej: `llama3.2:1b`)
- **messages** : lista completa de mensajes (contexto de la conversación)
- **stream=True** : activa respuesta en streaming

for chunk in response : Itera sobre los fragmentos que llegan. Cada **chunk** contiene:

- **choices[0].delta.content** : el texto del token actual (puede ser `None`)
- **choices[0].finish_reason** : `"stop"` cuando termina

print(content_chunk, end="") : Imprime sin salto de línea, para que los tokens aparezcan uno tras otro en la misma línea.

3.3.5 Guardar respuesta (línea 47)

```
messages.append({"role": "assistant", "content": assistant_reply})
```

Se agrega la respuesta completa al historial, para que el modelo tenga contexto en la siguiente pregunta.

3.3.6 Manejo de errores (líneas 48-52)

```
except APIConnectionError:
    print("\n[red]Error: No se pudo conectar a Ollama.[/red]")
    print("[yellow]Asegúrate de que Ollama esté corriendo:[/yellow]")
```

```
print("  ollama serve")
break
```

`APIConnectionError` se lanza cuando el SDK no puede conectar con el servidor. En lugar de un crash con traceback feo, se muestra un mensaje amigable.

3.4 Punto de entrada (líneas 55-56)

```
if __name__ == "__main__":
    typer.run(main)
```

`if __name__ == "__main__"` : Patrón estándar de Python. El código dentro solo se ejecuta cuando el archivo se corre directamente (`python main.py`), no cuando se importa desde otro módulo.

`typer.run(main)` : Toma la función `main()` y la convierte en un comando CLI. Typer automáticamente:

- Genera ayuda con `--help`
 - Maneja argumentos de línea de comandos (si los hubiera)
 - Ejecuta la función
-

4. Desglose de cada archivo

4.1 `main.py`

Propósito: Único archivo ejecutable. Contiene toda la lógica. **Líneas:** 56

Responsabilidades:

- Configuración (líneas 8-12)
- Interfaz de usuario (líneas 16-20)
- Bucle de conversación (líneas 26-52)
- Manejo de errores (líneas 48-52)

4.2 `requirements.txt`

Propósito: Lista de dependencias para `pip install`. **openai>=1.0.0:** SDK para comunicarse con la API. **typer>=0.9.0:** Framework CLI. **rich>=13.0.0:** Formato de terminal.

4.3 `config.example.py`

Propósito: Documentar las variables de entorno disponibles. **Uso:** Copiar a `.env` o exportar las variables manualmente. **Nota:** En la versión actual, el código usa `os.getenv()` directamente en `main.py` en lugar de leer este archivo.

4.4 .gitignore

Propósito: Evitar que archivos locales se suban a Git. **Excluye:**

- `.venv/` — entorno virtual (pesado y específico de cada máquina)
- `config.py` — archivo con claves reales (evitar fugas de secrets)
- `__pycache__/` y `*.pyc` — archivos compilados de Python
- `.vscode/` y `.idea/` — configuraciones de IDE

4.5 README.md

Propósito: Documentación del proyecto para quien visite el repositorio.

5. Dependencias externas

5.1 Ollama (no está en requirements.txt)

Ollama es un **servidor** que debe instalarse aparte. No es una librería Python.

Instalación: `curl -fsSL https://ollama.ai/install.sh | sh`

Inicio: `ollama serve` (corre en segundo plano en `http://localhost:11434`)

Modelos: Se descargan con `ollama pull <nombre>` (ej: `llama3.2:1b` , `llama3.1:8b` , `mistral` , `codellama`)

5.2 openai (librería Python)

SDK oficial de OpenAI. Su función principal es `client.chat.completions.create()` que:

1. Conecta a `base_url/v1/chat/completions` (endpoint REST)
2. Envía un JSON con `model` , `messages` , `stream`
3. Recibe la respuesta (completa o en streaming)

Aunque Ollama no es OpenAI, implementa el mismo protocolo REST, por lo que el SDK funciona sin modificaciones.

5.3 typer

Construido sobre Click. Convierte funciones Python en comandos CLI:

- `typer.run(main)` — ejecuta `main()` como comando
- `@app.command()` — para múltiples comandos
- Soporta argumentos, opciones, `--help` automático

5.4 rich

Provee:

- `print()` mejorado con formato BBCode (`[bold]` , `[red]` , `[yellow]` , etc.)
 - `Table()` con columnas, bordes, alineación
 - `Panel()` , `Progress()` , `Markdown()` , `Syntax()`
-

6. Conceptos clave

6.1 Streaming vs no-streaming

| Sin streaming | Con streaming | |---|---| | Esperas todo el tiempo de generación | Ves los tokens en tiempo real | | La respuesta llega completa de una vez | La respuesta llega fragmentada | | Menor complejidad de código | Mayor complejidad (bucle) | | Peor experiencia de usuario | Mejor experiencia de usuario |

6.2 El protocolo de mensajes

El formato de mensajes es un estándar en APIs de LLMs:

```
[
  {"role": "system", "content": "Eres un asistente..."},
  {"role": "user", "content": "¿Qué es Python?"},
  {"role": "assistant", "content": "Python es un lenguaje..."},
  {"role": "user", "content": "¿Y qué es un decorador?"}
]
```

Cada interacción agrega **dos mensajes**: la pregunta del usuario y la respuesta del asistente. El modelo usa todo el historial para generar contexto.

6.3 Variables de entorno

`OLLAMA_URL=http://localhost:11434/v1` — apunta a Ollama local. Si se cambia a `https://api.openai.com/v1` y se pone una API key real, el mismo código funciona con OpenAI.

7. Posibles mejoras

7.1 Inmediatas (bajo esfuerzo, alto impacto)

- Separar configuración en `config.py`
- Agregar `--help` personalizado con Typer
- Guardar conversaciones en JSON

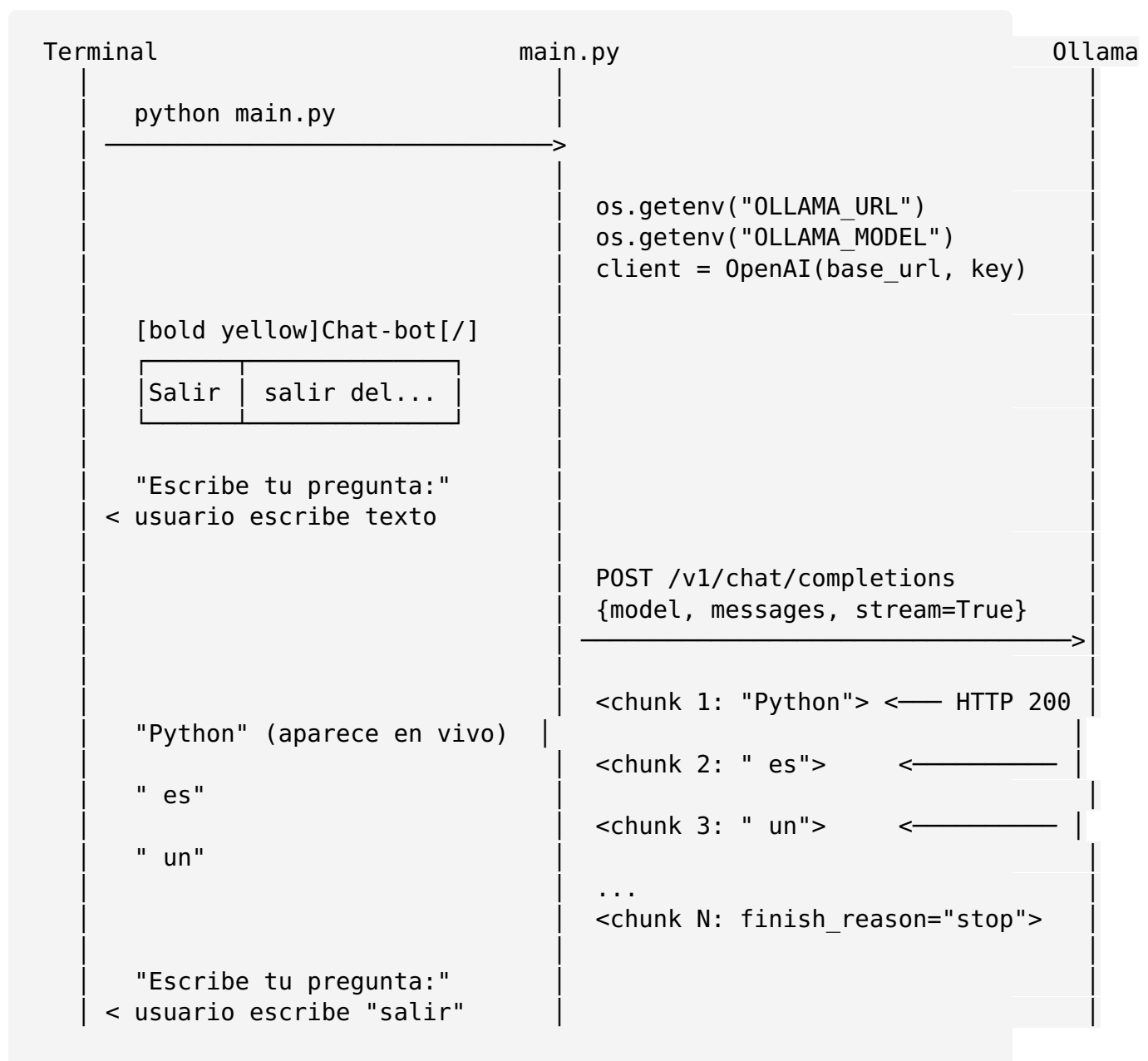
7.2 Mediano plazo

- Selector de modelos (flag `--model`)
- Múltiples conversaciones con nombres
- Exportar a Markdown

7.3 Avanzado

- Interfaz web con Gradio
- Plugins (ej: ejecutar código, buscar web)
- RAG (búsqueda en documentos propios)

8. Diagrama visual del flujo



Fin del programa